

DWH Schema Design (Part 2)

Learning Objectives

- Declare fact grain, keys, measures, and aggregation rules
- Choose star, snowflake, or galaxy schemas by workload
- Model history, table behavior, and conformed dimensions
- Design partitioning and governance controls for production use

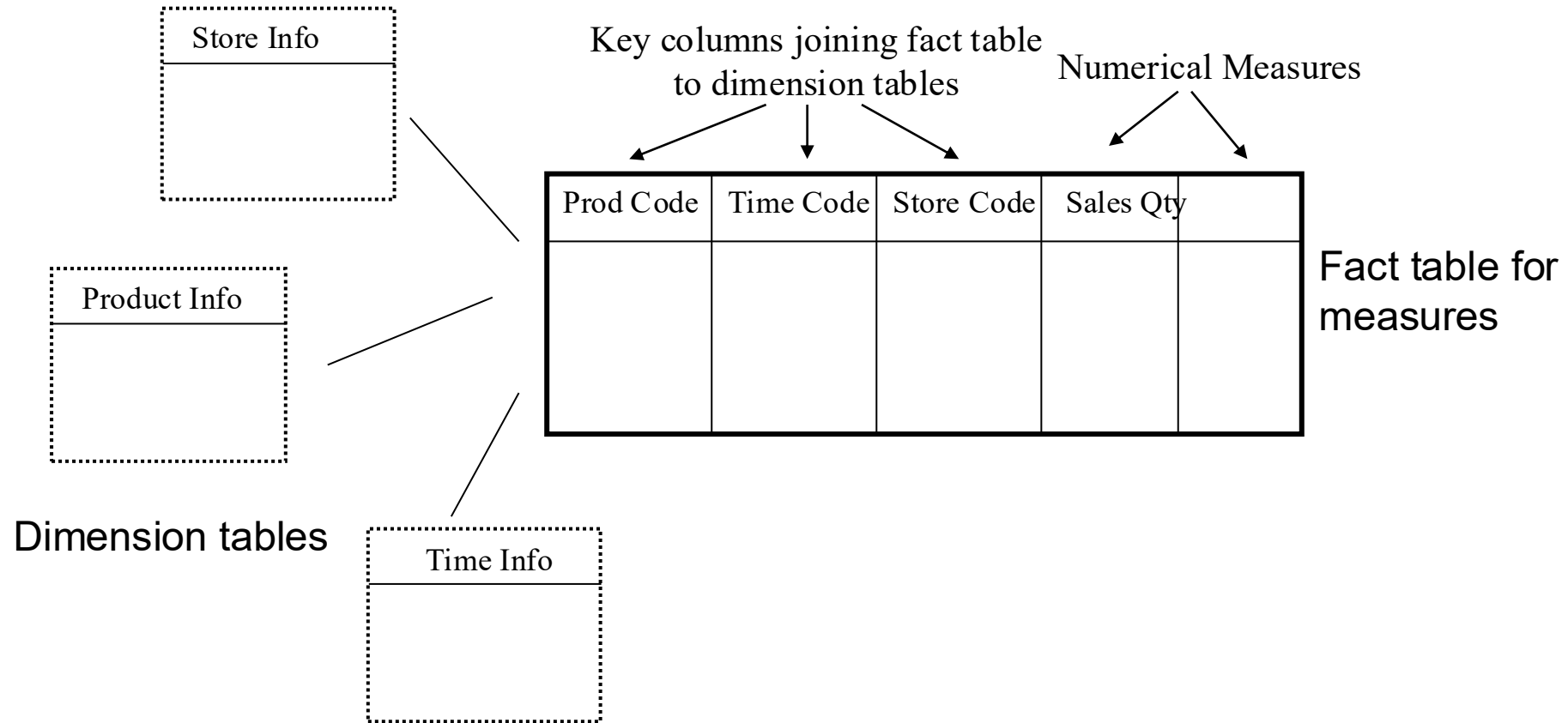
Why Schema Design Matters

- Schema design is where business meaning becomes executable
- Wrong grain or keys silently corrupt KPIs and downstream models
- Physical choices such as partitioning determine cost and latency
- Professional DWH design balances correctness, performance, and operability

Fact and Dimension Contracts

- Fact table: measurable business events at one declared grain
- Dimension table: descriptive context used to filter, group, and explain facts
- Measure: numeric value with explicit aggregation semantics
- Hierarchy: governed drill path such as day -> month -> quarter -> year

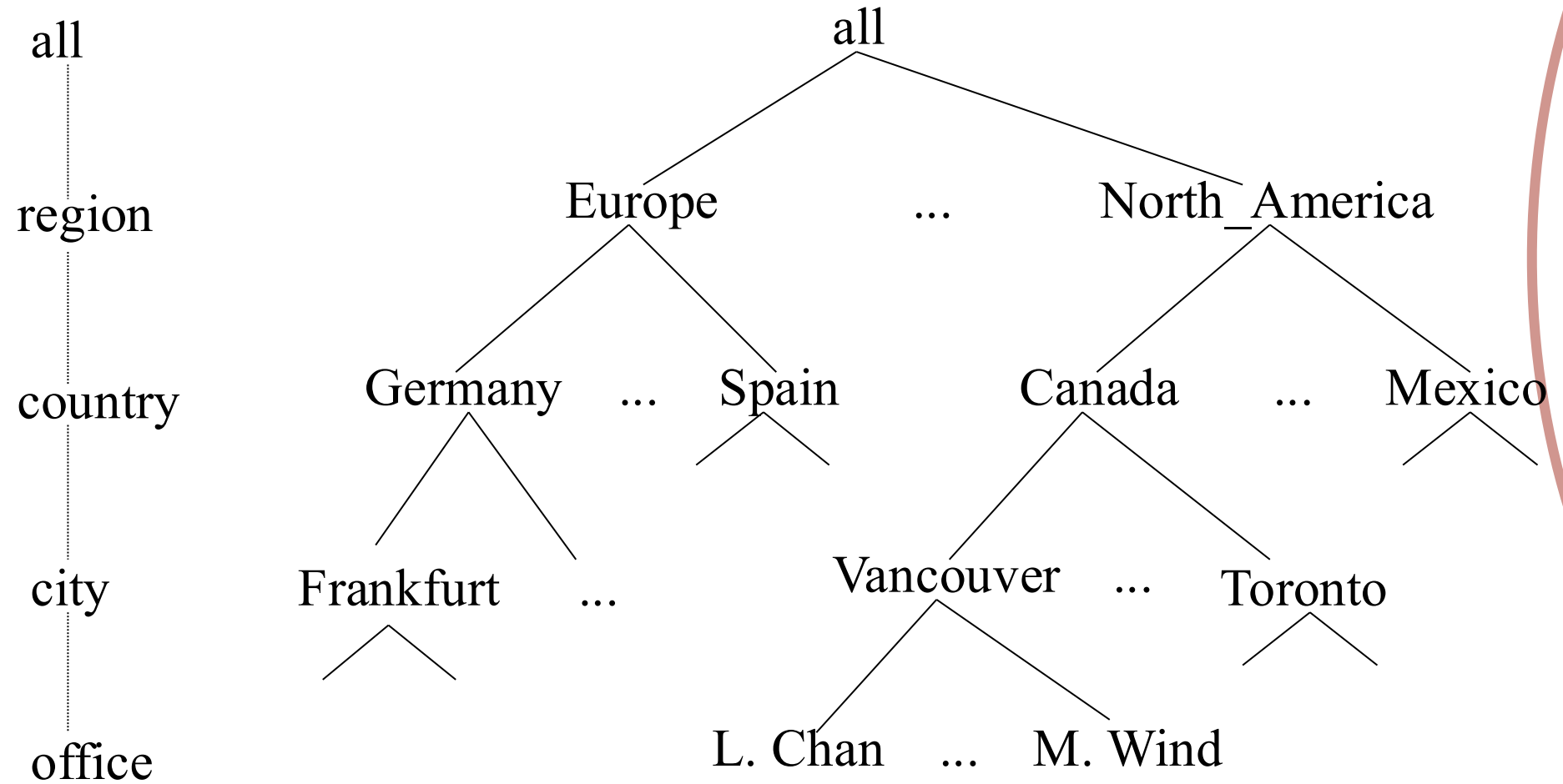
The Multi-Dimensional Model



Dimensional Modeling

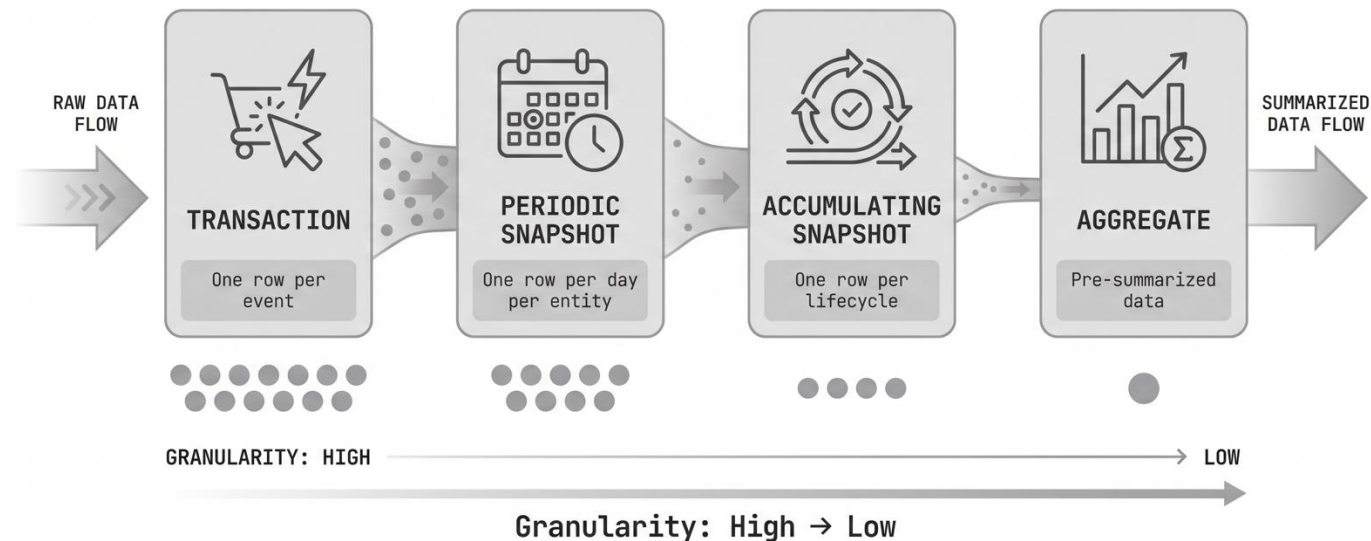
- Dimensions organize attributes into stable analysis paths
- E.g., Time: day -> week -> month -> quarter -> year
- E.g., Product: SKU -> product line -> brand -> category
- Conformed dimensions reuse the same keys and labels across facts
- Role-playing dimensions reuse one table in different roles, e.g., order_date and ship_date

Concept Hierarchy: Location Dimension



Declaring Fact Grain

- Grain defines exactly what one row in the fact table represents
- **If the grain is ambiguous, double counting is likely**
- Document grain and natural uniqueness before writing ETL



Grain: What It Is

- The grain is the business event level captured by the fact table
- It answers the question: what does one row mean?
- Changing grain later usually breaks joins, aggregates, and history

Common Grain Options

- Transaction grain: one row per event
- Periodic snapshot grain: one row per entity per period
- Accumulating snapshot grain: one row per process lifecycle
- Aggregate grain: pre-summarized rows
- Factless grain: event existence only

Choosing the Right Grain

- Start with the lowest atomic grain needed for analysis
- Use aggregate grains only for performance measures
- Avoid mixing grains in one fact table
- Check that the proposed grain has a stable unique key
- Write the grain statement before designing dimensions

Table Behavior Types

- Append-only
 - tables only insert new rows and preserve full event history
- Mutable tables
 - allow updates and deletes when the latest state matters
- Late-arriving
 - data reaches after the business event happened

Append-Only vs. Mutable Tables

- Append-only
 - best for logs, transactions, CDC history, and replayability
- Mutable
 - best for current-state dimensions, operational mirrors

A common pattern is append-only raw layer
plus curated mutable serving tables

Measure Types and Aggregation Rules

- Additive: safe to sum across all dimensions
- Semi-additive: sum across product, not across time
- Non-additive: calculate from numerator and denominator

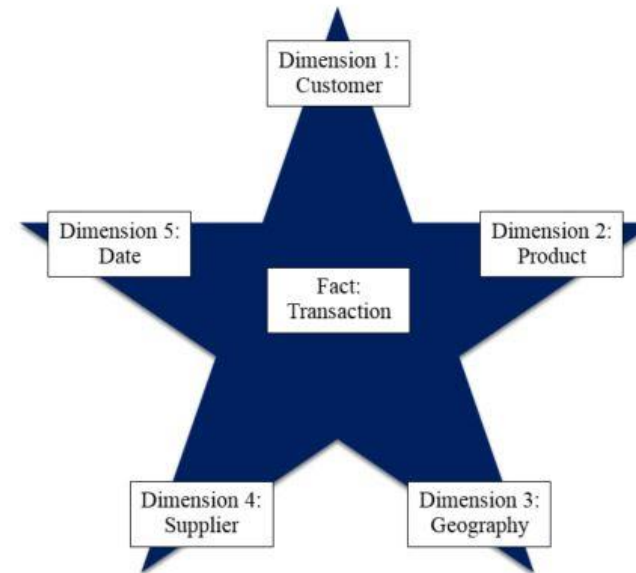
Common Schemas

Star Schema: Default BI Design



The Classic Star Schema

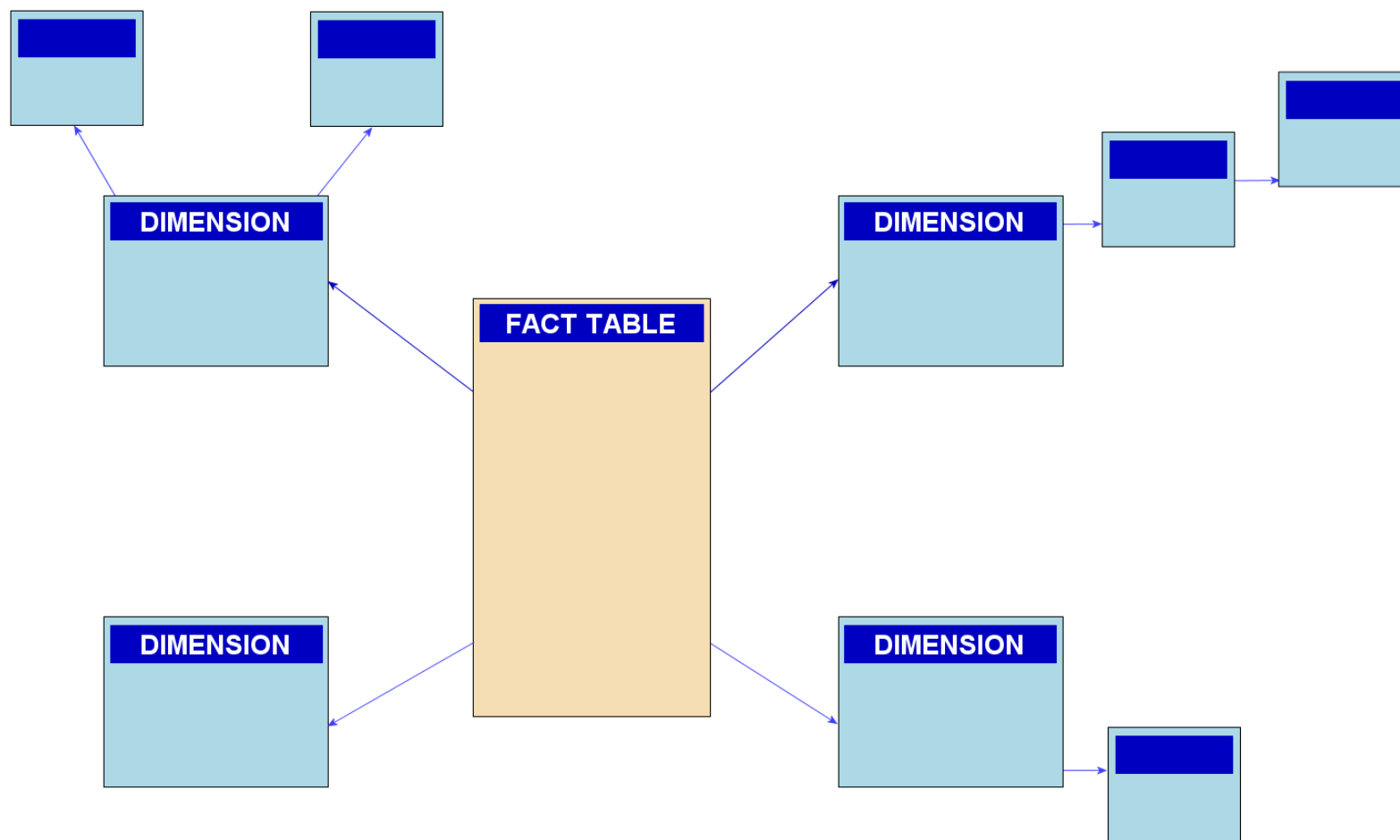
- A fact table sits at the center and connects to denormalized dimensions.
- Each dimension key appears once in the fact table primary/unique key.



Snowflake Schema

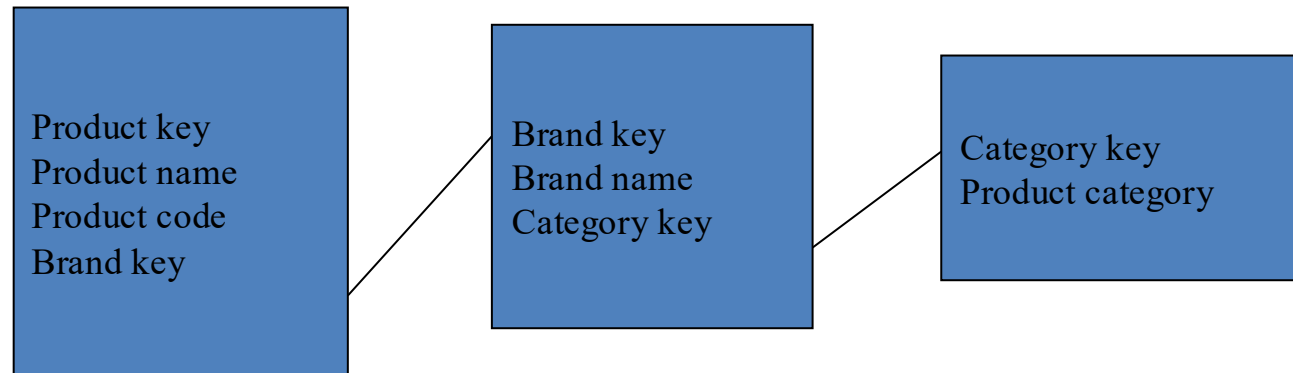
- Snowflaking normalizes selected dimension attributes into sub-dimensions
- It can reduce redundancy in large dimensions
- It adds joins, ETL complexity, and harder self-service BI
- Common compromise: normalized core plus star marts

Snowflake Schema: Normalized Dimensions



Snowflaking a Product Dimension

- Split shared hierarchy attributes into reference tables
- Product stores brand_key and category_key
- Benefit: less repetition and stronger governance
- Cost: extra joins unless a mart denormalizes back



Partitioning and Clustering Strategy

- Partition large facts by the dominant time filter, usually date_key
- Align partition granularity with query patterns and retention policy
- Cluster or sort on common filters such as customer_key or product_key
- Avoid tiny partitions and small files; they add metadata overhead
- Validate design with bytes scanned, partitions read, and p95 latency

Partition Pruning Cost Model

$$\text{ScanCost} = s \cdot |F|$$

$|F|$: total fact size

s : selected partition fraction

Example: $|F| = 1$ TB, one-day filter over 365 days ->

$s = 1/365$ - Scan drops from 1 TB to about 2.74 GB,

Cost and runtime often drop proportionally

Join Cost Intuition

$$\text{JoinWork} \approx O(|F_{\text{pruned}}| + \sum |D_i|)$$

Pruned fact size is usually dominant term

Broadcast joins help when dimensions fit memory

Large dimensions force shuffle and spill